RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105



CS23331 - DESIGN AND ANALYSIS OF ALGORITHMS

Laboratory Record Note Book

Name :. **RITHIKKA P** …… ……

Year / Branch / Section : .**II-Year/Computer Science and Engineering/F**

University Register No. : **2116250701601** ……

College Roll No. : .**250701601** ……

Semester : .**03** …… ……

Academic Year : .**2026-2027** ……

INDEX PAGE

Exp.No	Name of the Experiment	Date of Completion

Finding Time Complexity - Counter Method

Convert each algorithm into a program and find its time complexity using the counter method.

Question 1

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void function (int n)
{
    int i = 1;
    int s = 1;
    while (s <= n)
    {
        i++;
        s += i;
    }
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.
Input: A positive Integer n
Output: Print the value of the counter variable

For example:

Input	Result
9	12

Answer (program)

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);

    int i = 1;
    int s = 1;
    int count = 2;

    while (1)
    {
        count++;

        if (s > n)
            break;

        count++;
        i++;

        count++;
        s += i;
    }

    printf("%d", count);

    return 0;
}
```

Sample run

Input: **9** Output (counter value): **12**

Time complexity

s takes the values 1, 3, 6, 10, ... i.e. s = i(i+1)/2. The loop stops when s > n, so i is about sqrt(2n). The loop body runs about sqrt(2n) times with a constant number of counter increments each time.

Time complexity: O(sqrt(n))

Question 2

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void func(int n)
{
    if(n==1)
    {
        printf("*");
    }
    else
    {
        for(int i=1; i<=n; i++)
        {
            for(int j=1; j<=n; j++)
            {
                printf("*");
                printf("*");
                break;
            }
        }
    }
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.
Input: A positive Integer n
Output: Print the value of the counter variable

Answer (program)

```
#include<stdio.h>

int count=0;
void func(int n)
{
    if(n==1)
    {
        count++;
    }
    else
    {
        for(int i=1; i<=n; i++)
        {
            count++;
            for(int j=1; j<=n; j++)
            {
                count++;
                count++;
                count++;

                break;
            }
            count++;
        }
        count++;
    }
    count++;
    printf("%d",count);
}
int main(){
    int n;

    scanf("%d",&n);

    func(n);

    return 0;
}
```

Sample run

Input: **5** Output (counter value): **27**

Time complexity

The inner loop executes only once per outer iteration because of the break statement. So the outer loop runs n times and each iteration does a constant amount of work.

Time complexity: O(n)

Question 3

Convert the following algorithm into a program and find its time complexity using the counter method.

```
Factor(num) {
{
    for (i = 1; i <= num; ++i)
    {
        if (num % i == 0)
        {
            printf("%d ", i);
        }
    }
}
```

Note: No need of counter increment for declarations and scanf() and counter variable printf() statement.
Input: A positive Integer n
Output: Print the value of the counter variable

Answer (program)

```
#include<stdio.h>

void Factor(int num)
{
    int count = 0;
    for (int i = 1; i <= num; ++i)
    {
        count++;
        count++;

        if (num % i == 0)
        {
            count++;
        }
    }
    count++;
    printf("%d", count);
}

int main()
{
    int n;
    scanf("%d",&n);

    Factor(n);

    return 0;
}
```

Sample run

Input: **6** Output (counter value): **17**

Time complexity

The loop runs from 1 to num, so it executes num times. Inside it there is one modulus check (and an occasional print) which costs constant time.

Time complexity: O(n)

Question 4

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void function(int n)
{
    int c = 0;
    for(int i=n/2; i<n; i++)
        for(int j=1; j<n; j = 2 * j)
            for(int k=1; k<n; k = k * 2)
                c++;
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.
Input: A positive Integer n
Output: Print the value of the counter variable

Answer (program)

```
#include<stdio.h>

void function(int n)
{
    int c= 0;
    for(int i=n/2; i<n; i++){
        c++;

        for(int j=1; j<n; j = 2 * j){
            c++;

            for(int k=1; k<n; k = k * 2){
                c++;
                c++;

            }
            c++;
        }
        c++;
    }
    c++;
    c++;

    printf("%d",c);
}
int main()
{
    int n;
    scanf("%d",&n);

    function(n);

    return 0;
}
```

Sample run

Input: **8** Output (counter value): **106**

Time complexity

The outer loop (i from n/2 to n) runs n/2 times. The j loop doubles each time, so it runs log n times. The k loop also doubles each time, so it runs log n times. Total work = (n/2) x log n x log n.

Time complexity: $O(n \log^2 n)$

Question 5

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void reverse(int n)
{
    int rev = 0, remainder;
    while (n != 0)
    {
        remainder = n % 10;
        rev = rev * 10 + remainder;
        n /= 10;
    }
    print(rev);
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.
Input: A positive Integer n
Output: Print the value of the counter variable

Answer (program)

```
#include<stdio.h>

void reverse(int n)
{
    int rev = 0, remainder;
    int c=0;
    while (n != 0) {
        remainder = n % 10;
        rev = rev * 10 + remainder;
        n/= 10;
        c+=4;
    }
    c+=3;
    printf("%d",c);

}

int main()
{
    int n;
    scanf("%d",&n);

    reverse(n);

    return 0;
}
```

Sample run

Input: **1234** Output (counter value): **19**

Time complexity

Each iteration removes one digit of n (n is divided by 10). A number n has about log10(n) digits, so the loop runs log10(n) times with constant work per iteration.

Time complexity: O(log n)

Greedy Algorithms

Questions and Answers

Question 1

Problem Statement

Write a program to take value V and we want to make change for V Rs, and we have infinite supply of each of the denominations in Indian currency, i.e., we have infinite supply of { 1, 2, 5, 10, 20, 50, 100, 500, 1000} valued coins/notes, what is the minimum number of coins and/or notes needed to make the change.

Input Format: Take an integer from stdin.

Output Format: Print the integer which is change of the number.

Explanation: For input 64 we need a 50 Rs note, a 10 Rs note and two 2 rupee coins, so the output is 4.

Example

Input	Output
64	4

Answer (C program)

```
#include <stdio.h>

int main()
{
    int v, count = 0;
    int coins[] = {1000, 500, 100, 50, 20, 10, 5, 2, 1};

    scanf("%d", &v);

    for(int i = 0; i < 9; i++)
    {
        count = count + v / coins[i];
        v = v % coins[i];
    }

    printf("%d", count);

    return 0;
}
```

Output of the program

Input	Output
64	4

Greedy Approach

1. Keep the denominations in descending order: 1000, 500, 100, 50, 20, 10, 5, 2, 1.
2. For each denomination, add (V / coin) to the count.
3. Replace V with (V % coin) and continue with the next smaller denomination.

Time Complexity

Time complexity of this program: O(1)
The loop runs over a fixed set of 9 denominations (O(d) for d denominations).

Question 2

Problem Statement

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor g[i], which is the minimum size of a cookie that the child will be content with; and each cookie j has a size s[j]. If s[j] >= g[i], we can assign the cookie j to the child i, and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Input Format: First line: number of children n. Second line: n greed factors. Third line: number of cookies m. Fourth line: m cookie sizes.

Output Format: Print the maximum number of content children.

Explanation: You have 3 children and 2 cookies. The greed factors are 1, 2, 3. Both cookies have size 1, so only the child with greed factor 1 can be content. The output is 1.

Constraints: 1 <= g.length <= $3 * 10^4$ 0 <= s.length <= $3 * 10^4$ 1 <= g[i], s[j] <= $2^{31} - 1$

Example

Input	Output
3 1 2 3 2 1 1	1

Answer (C program)

```
#include <stdio.h>

int main()
{
    int n, m;
    scanf("%d", &n);

    int g[n];
    for(int i = 0; i < n; i++)
        scanf("%d", &g[i]);

    scanf("%d", &m);

    int s[m];
    for(int i = 0; i < m; i++)
        scanf("%d", &s[i]);

    int count = 0;

    for(int i = 0; i < n; i++)
    {
        for(int j = i + 1; j < n; j++)
        {
            if(g[i] > g[j])
            {
                int temp = g[i];
```

```
                g[i] = g[j];
                g[j] = temp;
            }
        }
    }

    for(int i = 0; i < m; i++)
    {
        for(int j = i + 1; j < m; j++)
        {
            if(s[i] > s[j])
            {
                int temp = s[i];
                s[i] = s[j];
                s[j] = temp;
            }
        }
    }

    int i = 0, j = 0;

    while(i < n && j < m)
    {
        if(s[j] >= g[i])
        {
            count++;
            i++;
        }
        j++;
    }

    printf("%d", count);

    return 0;
}
```

Output of the program

Input	Output
3 1 2 3 2 1 1	1

Greedy Approach

1. Sort the greed factors g[] and the cookie sizes s[] in ascending order.
2. Use two pointers i (child) and j (cookie), both starting at 0.
3. If s[j] >= g[i], give the cookie to the child: count++ and i++. In every case move to the next cookie: j++.
4. Stop when children or cookies run out; count is the answer.

Time Complexity

Time complexity of this program: $O(n^2 + m^2)$
The two exchange sorts cost $O(n^2)$ and $O(m^2)$. The two-pointer matching is only O(n + m); with an efficient sort the total is O(n log n + m log m).

Question 3

Problem Statement

A person needs to eat burgers. Each burger contains a count of calorie. After eating the burger, the person needs to run a distance to burn out his calories. If he has eaten i burgers with c calories each, then he has to run at least 3^i * c kilometers to burn out the calories. For example, if he ate 3 burgers with the count of calorie in the order: [1, 3, 2], the kilometers he needs to run are (3^0 * 1) + (3^1 * 3) + (3^2 * 2) = 1 + 9 + 18 = 28.

But this is not the minimum, so need to try out other orders of consumption and choose the minimum value. Determine the minimum distance he needs to run. Note: He can eat burger in any order and use an efficient sorting algorithm. Apply greedy approach to solve the problem.

Input Format: First line contains the number of burgers. Second line contains calories of each burger which is n space-separated integers.

Output Format: Print the minimum number of kilometers needed to run to burn out the calories.

Sample: Input: 3 / 5 10 7 Output: 76

Another test: Input: 3 / 1 3 2 Output: 18

Example

Input	Output
3 5 10 7	76

Answer (C program)

```
#include <stdio.h>
#include <math.h>

int main() {
    int n;
    scanf("%d", &n);

    int arr[n];
    for(int i = 0; i < n; i++){
        scanf("%d",&arr[i]);
    }
    for(int i = 0; i < n; i++){
        for(int j = 0; j < n-1; j++){
            if(arr[j] < arr[j + 1]){
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
    int s = 0;
    for(int i = 0; i < n; i++){
        s+= (pow(n, i))*arr[i];
    }
    printf("%d",s);
    return 0;
}
```

Output of the program

Input	Output
3 5 10 7	76

Greedy Approach

1. The multiplier grows with every burger eaten (position 0, 1, 2, ...).
2. To keep the total small, eat the burger with the highest calories first and the lowest calories last.
3. Sort the calories in descending order and add arr[i] times the multiplier for position i (the test cases use n as the base of the multiplier).

Time Complexity

Time complexity of this program: $O(n^2)$
The bubble sort costs $O(n^2)$; the sum is O(n). With an efficient sort (merge sort / qsort) the total is O(n log n).

Question 4

Problem Statement

Given an array of N integer, we have to maximize the sum of arr[i] * i, where i is the index of the element (i = 0, 1, 2, ..., N). Write an algorithm based on Greedy technique with a Complexity O(nlogn).

Input Format: First line specifies the number of elements - n. The next n lines contain the array elements.

Output Format: Maximum Array Sum to be printed.

Example

Input	Output
5 2 5 3 4 0	40

Answer (C program)

```
#include <stdio.h>

int main()
{
    int n, i, j, temp;
    int arr[100];
    int sum = 0;

    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }


    for(i = 0; i < n; i++)
    {
        for(j = i + 1; j < n; j++)
        {
            if(arr[i] > arr[j])
            {
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }

    for(i = 0; i < n; i++)
    {
        sum = sum + arr[i] * i;
    }

    printf("%d", sum);

    return 0;
}
```

Output of the program

Input	Output
5 2 5 3 4 0	40

Greedy Approach

1. The index i is the multiplier, so the largest values should sit at the largest indices.
2. Sort the array in ascending order.
3. Compute the sum of arr[i] * i.

Time Complexity

Time complexity of this program: $O(n^2)$
The exchange sort costs $O(n^2)$; the sum is O(n). To meet the O(n log n) requirement in the question, use an O(n log n) sort such as merge sort or qsort.

Question 5

Problem Statement

Given two arrays array_One[] and array_Two[] of same size N. We need to first rearrange the arrays such that the sum of the product of pairs (1 element from each) is minimum. That is SUM (A[i] * B[i]) for all i is minimum.

Input Format: First line: n. Next n numbers: array A. Next n numbers: array B.

Output Format: Print the minimum sum of products.

Example

Input	Output
3 1 2 3 4 5 6	28

Answer (C program)

```
#include <stdio.h>

int main() {
    int n, i, j, temp, sum = 0;
    scanf("%d", &n);

    int a[n], b[n];

    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for(i = 0; i < n; i++)
        scanf("%d", &b[i]);

    for(i = 0; i < n; i++) {
        for(j = i + 1; j < n; j++) {
            if(a[i] > a[j]) {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }

            if(b[i] > b[j]) {
                temp = b[i];
                b[i] = b[j];
                b[j] = temp;
            }
        }
    }

    for(i = 0; i < n; i++)
        sum += a[i] * b[n - 1 - i];

    printf("%d", sum);

    return 0;
}
```

Output of the program

Input	Output
3 1 2 3 4 5 6	28

Greedy Approach

1. To minimize the sum of products, pair small values with large values.

2. Sort A in ascending order and B in ascending order.

3. Pair A[i] with B[n-1-i] and add the products.

Time Complexity

Time complexity of this program: $O(n^2)$

The exchange sorts cost $O(n^2)$; the pairing sum is O(n). With an efficient sort the total is O(n log n).

Divide and Conquer

Questions and Answers

Question 1

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format: First Line Contains Integer m - Size of array. Next m lines Contains m numbers - Elements of an array.

Output Format: First Line Contains Integer - Number of zeroes present in the given array.

Answer (C program)

```
#include <stdio.h>

int countZero(int a[], int low, int high)
{
    if(low > high)
        return 0;

    if(a[low] == 0)
        return high - low + 1;

    int mid = (low + high) / 2;

    if(a[mid] == 1)
        return countZero(a, mid + 1, high);
    else
        return countZero(a, low, mid);
}

int main()
{
    int n;
    scanf("%d", &n);

    int a[n];

    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("%d", countZero(a, 0, n - 1));

    return 0;
}
```

Output of the program

Input	Output
5 1 1 1 0 0	2

Divide and Conquer Approach

1. Look at the middle element a[mid].
2. If a[mid] is 1, all zeroes must be on the right side, so search (mid + 1 .. high).
3. If a[mid] is 0, the first zero is at mid or on the left side, so search (low .. mid).
4. If a[low] is already 0, every element from low to high is 0, so return high - low + 1.

Time Complexity

Time complexity: O(log n)
Each call discards half of the array, like binary search (a simple linear scan would be O(n)).

Question 2

Problem Statement

Given an array nums of size n, return the majority element.

The majority element is the element that appears more than floor(n / 2) times. You may assume that the majority element always exists in the array.

Example 1: Input: nums = [3,2,3] Output: 3

Example 2: Input: nums = [2,2,1,1,1,2,2] Output: 2

Constraints: n == nums.length $1 <= n <= 5 * 10^4$ $-2^{31} <= nums[i] <= 2^{31} - 1$

Example

Input	Output
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer (C program)

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);

    int a[n];

    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    int candidate = a[0];
    int count = 1;

    for(int i = 1; i < n; i++)
    {
        if(a[i] == candidate)
            count++;
        else
            count--;

        if(count == 0)
        {
            candidate = a[i];
            count = 1;
        }
    }

    printf("%d", candidate);

    return 0;
}
```

Output of the program

Input	Output
7 2 2 1 1 1 2 2	2

Divide and Conquer Approach

1. Keep a candidate and a count. Start with candidate = a[0], count = 1.
2. For every next element: same as candidate - count++, different - count--.
3. When count becomes 0, pick the current element as the new candidate with count = 1.
4. Because the majority element appears more than half the time, it is the candidate left at the end.

Time Complexity

Time complexity: O(n)
One pass over the array with O(1) extra space (Boyer-Moore voting). This answer is not a divide and conquer solution; a divide and conquer version splits the array in two, finds the majority of each half and picks the winner, giving T(n) = 2T(n/2) + O(n) = O(n log n).

Question 3

Problem Statement

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format: First Line Contains Integer n - Size of array. Next n lines Contains n numbers - Elements of an array. Last Line Contains Integer x - Value for x.

Output Format: First Line Contains Integer - Floor value for x.

Answer (C program)

```
#include <stdio.h>

int floorValue(int a[], int low, int high, int x)
{
    if(low > high)
        return -1;

    int mid = (low + high) / 2;

    if(a[mid] == x)
        return a[mid];

    if(a[mid] < x)
    {
        int result = floorValue(a, mid + 1, high, x);

        if(result == -1)
            return a[mid];

        return result;
    }

    return floorValue(a, low, mid - 1, x);
}

int main()
{
    int n, x;
    scanf("%d", &n);

    int a[n];

    for(int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    scanf("%d", &x);

    printf("%d", floorValue(a, 0, n - 1, x));

    return 0;
}
```

Output of the program

Input	Output
5 1 2 8 10 12 7	2

Divide and Conquer Approach

1. Find the middle element a[mid].
2. If a[mid] == x, then x itself is the floor.
3. If a[mid] < x, a[mid] is a possible floor: search the right half. If the right half has no floor (-1), a[mid] is the answer.
4. If a[mid] > x, the floor lies in the left half: search (low .. mid - 1).
5. If the range becomes empty, return -1.

Time Complexity

Time complexity: O(log n)
Only one half is searched in each recursive call, like binary search. The program prints -1 when no element is less than or equal to x.

Question 4

Problem Statement

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format: First Line Contains Integer n - Size of array. Next n lines Contains n numbers - Elements of an array. Last Line Contains Integer x - Sum Value.

Output Format: First Line Contains Integer - Element1. Second Line Contains Integer - Element2 (Element 1 and Element 2 together sum to value "x").

Answer (C program)

```
#include <stdio.h>

int findPair(int a[], int left, int right, int x, int *p, int *q) {
    if (left >= right)
        return 0;

    int sum = a[left] + a[right];

    if (sum == x) {
        *p = a[left];
        *q = a[right];
        return 1;
    }

    if (sum < x)
        return findPair(a, left + 1, right, x, p, q);

    return findPair(a, left, right - 1, x, p, q);
}

int main() {
    int n, x, p, q;

    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    scanf("%d", &x);

    if (findPair(a, 0, n - 1, x, &p, &q)) {
        printf("%d\n%d", p, q);
    } else {
        printf("No");
    }

    return 0;
}
```

Output of the program

Input	Output
5 1 3 5 7 9 12	3 9

Divide and Conquer Approach

1. Take the smallest element a[left] and the largest element a[right] and find their sum.
2. If the sum equals x, print the two numbers.
3. If the sum is less than x, move left forward: recurse on (left + 1, right).
4. If the sum is greater than x, move right backward: recurse on (left, right - 1).
5. If left >= right, no pair exists: print No.

Time Complexity

Time complexity: O(n)
Each recursive call removes one element from the range, so at most n - 1 calls are made. Recursion depth is also O(n).

Question 5

Problem Statement

Write a Program to Implement the Quick Sort Algorithm.

Input Format: The first line contains the no of elements in the list - n. The next n lines contain the elements.

Output: Sorted list of elements

Example

Input	Output
5 67 34 12 98 78	12 34 67 78 98

Answer (C program)

```
#include <stdio.h>

void quickSort(int a[], int low, int high)
{
    if(low < high)
    {
        int i = low;
        int j = high;
        int pivot = a[low];

        while(i < j)
        {
            while(a[i] <= pivot && i < high)
                i++;

            while(a[j] > pivot)
                j--;

            if(i < j)
            {
                int temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }

        a[low] = a[j];
        a[j] = pivot;

        quickSort(a, low, j - 1);
        quickSort(a, j + 1, high);
    }
}

int main()
{
    int n;
    scanf("%d", &n);

    int a[n];
    for(int i = 0; i < n; i++)
```

```
        scanf("%d", &a[i]);

    quickSort(a, 0, n - 1);

    for(int i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

Output of the program

Input	Output
5 67 34 12 98 78	12 34 67 78 98

Divide and Conquer Approach

1. Divide: choose a[low] as the pivot and partition the array so that smaller elements are on its left and larger elements on its right (the pivot lands at index j).
2. Conquer: recursively sort the left part (low .. j - 1) and the right part (j + 1 .. high).
3. Combine: nothing to do, the array is sorted in place.

Time Complexity

Time complexity: Best / average case: O(n log n), worst case: O(n^2)
Partitioning takes O(n) per level. Balanced splits give log n levels. If the array is already sorted, the first-element pivot gives the worst case.

Dynamic Programming

Questions and Answers

Question 1 - Playing with Numbers

Problem Statement

Ram and Sita are playing with numbers by giving puzzles to each other. Now it was Ram term, so he gave Sita a positive integer 'n' and two numbers 1 and 3. He asked her to find the possible ways by which the number n can be represented using 1 and 3. Write any efficient algorithm to find the possible ways.

Example 1: Input: 6 Output: 6

Explanation: There are 6 ways to represent number 6 with 1 and 3: 1+1+1+1+1+1, 3+3, 1+1+1+3, 1+1+3+1, 1+3+1+1, 3+1+1+1.

Input Format: First Line contains the number n.

Output Format: Print: The number of possible ways 'n' can be represented using 1 and 3.

Example

Input	Output
6	6

Answer (C program)

```
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);

    long long dp[n + 1];

    for(int i = 0; i <= n; i++)
        dp[i] = 0;

    dp[0] = 1;

    for(int i = 1; i <= n; i++)
    {
        dp[i] = dp[i - 1];

        if(i >= 3)
            dp[i] += dp[i - 3];
    }

    printf("%lld", dp[n]);

    return 0;
}
```

Output of the program

Input	Output
6	6

Dynamic Programming Approach

1. Let dp[i] = number of ways to write i as an ordered sum of 1s and 3s.
2. Base case: dp[0] = 1 (one way to make 0: use nothing).
3. The last number used is either 1 or 3, so dp[i] = dp[i - 1] + dp[i - 3] (the second term only when i >= 3).
4. The answer is dp[n].

Time Complexity

Time complexity: O(n)
One loop from 1 to n with constant work per step. Space is O(n) for the dp array. (The plain recursion would be exponential.)

Question 2 - Playing with Chessboard

Problem Statement

Ram is given with an n*n chessboard with each cell with a monetary value. Ram stands at the (0,0), that the position of the top left white rook. He is been given a task to reach the bottom right black rook position (n-1, n-1) constrained that he needs to reach the position by traveling the maximum monetary path under the condition that he can only travel one step right or one step down the board. Help ram to achieve it by providing an efficient DP algorithm.

Example: Input: 3 / 1 2 4 / 2 3 4 / 8 7 1 Output: 19

Explanation: Totally there will be 6 paths among that the optimal is. Optimal path value: 1+2+8+7+1 = 19.

Input Format: First Line contains the integer n. The next n lines contain the n*n chessboard values.

Output Format: Print Maximum monetary value of the path

Example

Input	Output
3 1 2 4 2 3 4 8 7 1	19

Answer (C program)

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int a[n][n];
    int dp[n][n];

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
        }
    }

    dp[0][0] = a[0][0];

    for (int j = 1; j < n; j++)
        dp[0][j] = dp[0][j - 1] + a[0][j];

    for (int i = 1; i < n; i++)
        dp[i][0] = dp[i - 1][0] + a[i][0];

    for (int i = 1; i < n; i++) {
        for (int j = 1; j < n; j++) {
            dp[i][j] = a[i][j] + (dp[i - 1][j] > dp[i][j - 1] ? dp[i - 1][j] : dp[i][j - 1]);
        }
    }

    printf("%d", dp[n - 1][n - 1]);
```

```
    return 0;
}
```

Output of the program

Input	Output
3 1 2 4 2 3 4 8 7 1	19

Dynamic Programming Approach

1. Let dp[i][j] = maximum money collected on a path from (0,0) to cell (i,j).

2. First row: dp[0][j] = dp[0][j-1] + a[0][j] (only moves right). First column: dp[i][0] = dp[i-1][0] + a[i][0] (only moves down).

3. Every other cell is reached from above or from the left: dp[i][j] = a[i][j] + max(dp[i-1][j], dp[i][j-1]).

4. The answer is dp[n-1][n-1].

Time Complexity

Time complexity: $O(n^2)$

Every cell of the n x n board is filled once with constant work. Space is $O(n^2)$ for the dp table.

Question 3 - Longest Common Subsequence

Problem Statement

Given two strings find the length of the common longest subsequence (need not be contiguous) between the two. Solving it using Dynamic Programming.

Example: s1: ggtabe s2: tgatasb The length is 4 (the common subsequence is g, t, a, b).

For example: Input: aab / azb Result: 2

Example

Input	Output
aab azb	2

Answer (C program)

```
#include <stdio.h>
#include <string.h>

int main() {
    char s1[1000], s2[1000];

    scanf("%s", s1);
    scanf("%s", s2);

    int n = strlen(s1);
    int m = strlen(s2);

    int dp[n + 1][m + 1];

    for (int i = 0; i <= n; i++) {
        for (int j = 0; j <= m; j++) {
            if (i == 0 || j == 0) {
                dp[i][j] = 0;
            }
            else if (s1[i - 1] == s2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            }
            else {
                dp[i][j] = dp[i - 1][j] > dp[i][j - 1]
                           ? dp[i - 1][j]
                           : dp[i][j - 1];
            }
        }
    }

    printf("%d", dp[n][m]);

    return 0;
}
```

Output of the program

Input	Output
aab azb	2

Dynamic Programming Approach

1. Let dp[i][j] = length of the LCS of the first i characters of s1 and the first j characters of s2.
2. If i == 0 or j == 0, dp[i][j] = 0 (an empty string has no common subsequence).
3. If s1[i-1] == s2[j-1], the characters match: dp[i][j] = dp[i-1][j-1] + 1.
4. Otherwise dp[i][j] = max(dp[i-1][j], dp[i][j-1]).
5. The answer is dp[n][m].

Time Complexity

Time complexity: O(n * m)
n and m are the lengths of the two strings. The table has (n + 1) x (m + 1) cells, each filled in constant time. Space is also O(n * m).

Question 4 - Longest Non-decreasing Subsequence

Problem Statement

Find the length of the Longest Non-decreasing Subsequence in a given Sequence.

Eg: Input: 9 Sequence: [-1,3,4,5,2,2,2,2,3]

Explanation: the subsequence is [-1,2,2,2,2,3]

Output: 6

Example

Input	Output
9 -1 3 4 5 2 2 2 2 3	6

Answer (C program)

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int a[n], dp[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
        dp[i] = 1;
    }

    int ans = 1;

    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (a[j] <= a[i] && dp[j] + 1 > dp[i]) {
                dp[i] = dp[j] + 1;
            }
        }

        if (dp[i] > ans)
            ans = dp[i];
    }

    printf("%d", ans);

    return 0;
}
```

Output of the program

Input	Output
9 -1 3 4 5 2 2 2 2 3	6

Dynamic Programming Approach

1. Let dp[i] = length of the longest non-decreasing subsequence that ends at index i. Start with dp[i] = 1 (the element alone).
2. For every earlier index j < i with a[j] <= a[i], we can extend that subsequence: dp[i] = max(dp[i], dp[j] + 1).
3. The answer is the largest value in dp[] (kept in ans).

Time Complexity

Time complexity: $O(n^2)$
Two nested loops (i and j < i). Space is O(n) for dp[]. A faster O(n log n) solution exists using binary search, but this DP version is the standard one.

Competitive Programming

Questions and Answers

Question 1 - Find Duplicate in Array

Problem Statement

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format: First Line - Number of elements. n Lines - n Elements.

Output Format: Element x - That is repeated

Example

Input	Output
5 1 1 2 3 4	1

Solution 1 - Nested loops (no extra space)

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (a[i] == a[j]) {
                printf("%d", a[i]);
                return 0;
            }
        }
    }

    return 0;
}
```

Output of the program

Input	Output
5 1 1 2 3 4	1

Solution 2 - Counting array

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int count[n + 1];

    for (int i = 0; i <= n; i++)
        count[i] = 0;

    for (int i = 0; i < n; i++) {
        int x;
        scanf("%d", &x);
        count[x]++;
    }

    for (int i = 1; i <= n; i++) {
        if (count[i] > 1) {
            printf("%d", i);
            break;
        }
    }

    return 0;
}
```

Output of the program

Input	Output
5 1 1 2 3 4	1

Approach

Solution 1: compare every element a[i] with all later elements a[j]. The first pair with a[i] == a[j] gives the repeated number. The array is only read, never changed, and no extra array is used.

Solution 2: since every value is between 1 and n, use count[value] as a frequency table. Read all elements and do count[x]++. Then scan values 1 to n and print the first value whose count is greater than 1.

Time Complexity

Time complexity: Solution 1: O(n^2) time, O(1) space | Solution 2: O(n) time, O(n) space
Solution 1 needs two nested loops over the array. Solution 2 needs one pass to count and one pass to search, but it uses an extra count[] array of size n + 1.

Question 2 - Intersection of Two Sorted Arrays

Problem Statement

Find the intersection of two sorted arrays.

OR in other words, Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format: The first line contains T, the number of test cases. Following T lines contain: (1) Line 1 contains N1, followed by N1 integers of the first array. (2) Line 2 contains N2, followed by N2 integers of the second array.

Output Format: The intersection of the arrays in a single line

Example 1: Input: 1 / 3 10 17 57 / 6 2 7 10 15 57 246 Output: 10 57

Example 2: Input: 1 / 6 1 2 3 4 5 6 / 2 1 6 Output: 1 6

Example

Input	Output
1 3 10 17 57 6 2 7 10 15 57 246	10 57

Answer (C program)

```
#include <stdio.h>

int main() {
    int T;
    scanf("%d", &T);

    while (T--) {
        int n1, n2;
        scanf("%d", &n1);

        int a[n1];
        for (int i = 0; i < n1; i++)
            scanf("%d", &a[i]);

        scanf("%d", &n2);

        int b[n2];
        for (int i = 0; i < n2; i++)
            scanf("%d", &b[i]);

        int i = 0, j = 0;

        while (i < n1 && j < n2) {
            if (a[i] == b[j]) {
                printf("%d ", a[i]);
                i++;
                j++;
            }
            else if (a[i] < b[j]) {
                i++;
            }
            else {
                j++;
```

```
            }
        }

        printf("\n");
    }

    return 0;
}
```

Output of the program

Input	Output
1 3 10 17 57 6 2 7 10 15 57 246	10 57

Approach

1. Use two pointers: i on the first array and j on the second array, both starting at 0.
2. If a[i] == b[j], the element is in both arrays: print it and move both pointers.
3. If a[i] < b[j], move i forward. Otherwise move j forward (both arrays are sorted, so the smaller value cannot match anything later).
4. Stop when either array is finished, then print a new line for the next test case.

Time Complexity

Time complexity: O(N1 + N2) per test case
Each pointer only moves forward, so at most N1 + N2 steps are made. No extra space is needed apart from the input arrays.

Question 3 - Pair with Difference k

Problem Statement

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format: First Line n - Number of elements in an array. Next n Lines - N elements in the array. k - Non - Negative Integer.

Output Format: 1 - If pair exists. 0 - If no pair exists.

Explanation for the given Sample Testcase: YES as 5 - 1 = 4. So Return 1.

Example

Input	Output
3 1 3 5 4	1

Answer (C program)

```
#include <stdio.h>

int main() {
    int n, k;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    scanf("%d", &k);

    int i = 0, j = 1;

    while (i < n && j < n) {
        if (i == j) {
            j++;
        }
        else if (a[j] - a[i] == k) {
            printf("1");
            return 0;
        }
        else if (a[j] - a[i] < k) {
            j++;
        }
        else {
            i++;
        }
    }

    printf("0");

    return 0;
}
```

Output of the program

Input	Output
3 1 3 5 4	1

Approach

1. Use two pointers i = 0 and j = 1 on the sorted array.
2. If i == j, move j forward so the two indices are different.
3. If a[j] - a[i] == k, a pair exists: print 1.
4. If the difference is smaller than k, move j forward to make it bigger. If it is bigger than k, move i forward to make it smaller.
5. If a pointer reaches the end without a match, print 0.

Time Complexity

Time complexity: O(n)
Both pointers only move forward, so there are at most 2n steps. A brute force check of every pair would be $O(n^2)$.